

Chapitre 6

Synthèse du Chapitre 6 : Des records aux collections bulk en PL/SQL

Ce chapitre explore l'utilisation des **records** et des **collections bulk** en PL/SQL, qui permettent de manipuler des structures de données complexes et d'optimiser les opérations sur les bases de données. Les records sont des structures hétérogènes (similaires aux structures en langage C), tandis que les collections bulk permettent de traiter plusieurs lignes de données en une seule opération, ce qui améliore les performances.

1. Déclarations de records

Les **records** permettent de regrouper plusieurs champs de types différents dans une seule structure. Ils sont utiles pour manipuler des lignes de tables ou des ensembles de données complexes.

- **Déclaration d'un record :**

- Syntaxe :

```
TYPE nom_type IS RECORD (  
    champ1 type1,  
    champ2 type2,  
    ...  
);
```

- Exemple :

```
TYPE TypeDept IS RECORD (  
    NumDep VARCHAR2(6),  
    NomDep VARCHAR2(40),  
    NumSecu VARCHAR2(8)  
);  
UnDepartement TypeDept;
```

- **Utilisation de `%ROWTYPE` :**

- Permet de créer un record basé sur la structure d'une table.

- Exemple :

```
UnDepartement Departements%ROWTYPE;
```

2. Initialisation et accès aux champs

Les champs d'un record sont accessibles via la notation pointée (`record.champ`).

- Exemple d'initialisation et d'accès :

```
DECLARE
  UnDepartement Departements%ROWTYPE;
BEGIN
  UnDepartement.NumDep := 'd00006';
  UnDepartement.NomDep := 'Inpres';
  UnDepartement.NumSecu := '192357';
  DBMS_OUTPUT.PUT_LINE('Nom département : ' || UnDepartement.NomDep);
END;
```

3. Comparaison de deux records

En PL/SQL, il n'est pas possible de comparer directement deux records. Il faut comparer chaque champ individuellement.

- Exemple de comparaison :

```
DECLARE
  UnDepartement Departements%ROWTYPE;
  UnDepartement2 Departements%ROWTYPE;
BEGIN
  IF UnDepartement.NumDep = UnDepartement2.NumDep
    AND UnDepartement.NomDep = UnDepartement2.NomDep
    AND UnDepartement.NumSecu = UnDepartement2.NumSecu THEN
    DBMS_OUTPUT.PUT_LINE('OK');
  ELSE
    DBMS_OUTPUT.PUT_LINE('NOK');
```

```
END IF;  
END;
```

4. Utilisation des records dans les opérations SQL

Les records peuvent être utilisés pour simplifier les opérations SQL comme

`INSERT`, `UPDATE`, et `SELECT`.

- **Insertion avec un record :**

```
DECLARE  
  UnDepartement Departements%ROWTYPE;  
BEGIN  
  UnDepartement.NumDep := 'd00006';  
  UnDepartement.NomDep := 'Inpres';  
  UnDepartement.NumSecu := '192357';  
  INSERT INTO Departements VALUES UnDepartement;  
  COMMIT;  
END;
```

- **Mise à jour avec un record :**

```
DECLARE  
  UnDepartement Departements%ROWTYPE;  
BEGIN  
  UnDepartement.NumDep := 'd00006';  
  UnDepartement.NomDep := 'Rennequin S';  
  UnDepartement.NumSecu := '451278';  
  UPDATE Departements SET ROW = UnDepartement WHERE NumDep  
= 'd00006';  
  COMMIT;  
END;
```

5. FORALL et collections bulk

La commande `FORALL` permet d'exécuter des opérations DML (`INSERT`, `UPDATE`, `DELETE`) en masse sur des collections, ce qui améliore les performances en réduisant le nombre de context switches entre PL/SQL et SQL.

- **Exemple de `FORALL` :**

```

DECLARE
    TYPE TableEmployes IS TABLE OF Employees%ROWTYPE INDEX BY
    BINARY_INTEGER;
    LesEmployes TableEmployes;
BEGIN
    -- Remplir la collection LesEmployes
    FORALL i IN LesEmployes.FIRST..LesEmployes.LAST
        INSERT INTO Employees VALUES LesEmployes(i);
    COMMIT;
END;

```

6. Gestion des exceptions dans FORALL

La gestion des exceptions dans `FORALL` peut se faire avec la clause `SAVE EXCEPTIONS`, qui permet de continuer l'exécution même si certaines opérations échouent.

- **Exemple de gestion des exceptions :**

```

DECLARE
    TYPE TableEmployes IS TABLE OF Employees%ROWTYPE INDEX BY
    BINARY_INTEGER;
    LesEmployes TableEmployes;
    errors EXCEPTION;
    PRAGMA EXCEPTION_INIT(errors, -24381);
BEGIN
    -- Remplir la collection LesEmployes
    FORALL i IN LesEmployes.FIRST..LesEmployes.LAST SAVE EXCEPTI
    ONS
        INSERT INTO Employees VALUES LesEmployes(i);
    EXCEPTION
        WHEN errors THEN
            DBMS_OUTPUT.PUT_LINE('Erreurs lors de l\\insertion');
END;

```

7. BULK COLLECT

La clause `BULK COLLECT` permet de charger plusieurs lignes d'une requête SQL dans une collection en une seule opération.

- **Exemple de `BULK COLLECT` :**

```
DECLARE
    TYPE TableEmployes IS TABLE OF Employees%ROWTYPE INDEX BY
    BINARY_INTEGER;
    LesEmployes TableEmployes;
BEGIN
    SELECT * BULK COLLECT INTO LesEmployes FROM Employees WH
    ERE NumDep = 'd00004';
    FOR i IN LesEmployes.FIRST..LesEmployes.LAST LOOP
        DBMS_OUTPUT.PUT_LINE(LesEmployes(i).Nom);
    END LOOP;
END;
```

8. Utilisation conjointe de **FORALL** et **BULK COLLECT**

Il est possible de combiner `FORALL` et `BULK COLLECT` pour optimiser les opérations de mise à jour et de récupération de données.

- **Exemple combiné :**

```
DECLARE
    TYPE TableEmployes IS TABLE OF Employees%ROWTYPE INDEX BY
    BINARY_INTEGER;
    LesEmployes TableEmployes;
BEGIN
    -- Récupérer les employés
    SELECT * BULK COLLECT INTO LesEmployes FROM Employees WH
    ERE NumDep = 'd00004';

    -- Mettre à jour les employés
    FORALL i IN LesEmployes.FIRST..LesEmployes.LAST
        UPDATE Employees SET Bareme = Bareme * 1.1 WHERE NumSecu
        = LesEmployes(i).NumSecu;

    COMMIT;
END;
```

Conclusion

Les **records** et les **collections bulk** sont des outils puissants en PL/SQL pour manipuler des structures de données complexes et optimiser les opérations sur les bases de données. Les records permettent de regrouper des champs hétérogènes, tandis que les collections bulk (via `FORALL` et `BULK COLLECT`) améliorent les performances en traitant plusieurs lignes en une seule opération. Une bonne compréhension de ces concepts est essentielle pour écrire des programmes PL/SQL efficaces et performants.